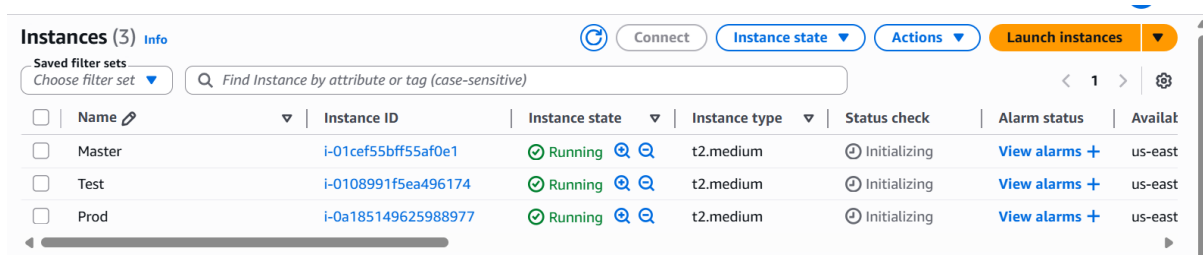


You have been hired as a Sr. DevOps Engineer in Abode Software. They want to implement DevOps Lifecycle in their company. You have been asked to implement this lifecycle as fast as possible. Abode Software is a product-based company and their product is available on this GitHub link. <https://github.com/hshar/webapp> Following are the specifications of the lifecycle: 1. Install the necessary software on the machines using a configuration management tool 2. Git workflow has to be implemented 3. CodeBuild should automatically be triggered once a commit is made to master branch or develop branch. a. If a commit is made to master branch, test and push to prod b. If a commit is made to develop branch, just test the product, do not push to prod 4. The code should be containerized with the help of a Dockerfile. The Dockerfile should be built every time there is a push to GitHub. Use the following pre-built container for your application: hshar/webapp The code should reside in '/var/www/html' 5. The above tasks should be defined in a Jenkins Pipeline with the following jobs: a. Job1 : build b. Job2 : test c. Job3 : prod

Live commands to perform:

Login to AWS Console:
Create 3 Machines(t2.medium)

On all Machines:
sudo apt update



The screenshot shows the AWS Management Console 'Instances' page. There are three instances listed: 'Master', 'Test', and 'Prod'. All three are in a 'Running' state. The 'Master' instance has ID 'i-01cef55bff55af0e1'. The 'Test' instance has ID 'i-0108991f5ea496174'. The 'Prod' instance has ID 'i-0a185149625988977'. All instances are of type 't2.medium' and are located in the 'us-east' region. The 'Status check' column shows 'Initializing' for all three. The 'Alarm status' column has a 'View alarms' link with a plus icon. The 'Availability' column shows 'us-east' for all three.

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability
☐	Master	i-01cef55bff55af0e1	Running	t2.medium	Initializing	View alarms +	us-east
☐	Test	i-0108991f5ea496174	Running	t2.medium	Initializing	View alarms +	us-east
☐	Prod	i-0a185149625988977	Running	t2.medium	Initializing	View alarms +	us-east

On Master Machine:
 sudo apt install software-properties-common
 sudo add-apt-repository --yes --update ppa:ansible/ansible
 sudo apt install ansible -y
 ansible --version
 ssh-keygen
 sudo cat ~/.ssh/id_rsa.pub
 Copy the output


```
ubuntu@ip-172-31-30-100:/etc/ansible$ ansible -m ping all
[WARNING]: Platform linux on host 172.31.24.74 is using the discovered Python interpreter at /usr/bin/python3.10, but future installation of another
Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html
for more information.
172.31.24.74 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.10"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Platform linux on host 172.31.31.138 is using the discovered Python interpreter at /usr/bin/python3.10, but future installation of another
Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.17/reference_appendices/interpreter_discovery.html
for more information.
172.31.31.138 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.10"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu@ip-172-31-30-100:/etc/ansible$
```

i-01cef55bff55af0e1 (Master)
PublicIPs: 3.80.44.25 PrivateIPs: 172.31.30.100

sudo nano play.yml

Paste the below content to the editor

```
---
- name: task for Master
  hosts: localhost
  become: true
  tasks:
    - name: executing Master Script
      script: master.sh

- name: task for Slaves
  hosts: Slaves
  become: true
  tasks:
    - name: executing Slave Script
      script: slave.sh
```

```
GNU nano 6.2 play.yml
---
- name: task for Master
  hosts: localhost
  become: true
  tasks:
    - name: executing Master Script
      script: master.sh

- name: task for Slaves
  hosts: Slaves
  become: true
  tasks:
    - name: executing Slave Script
      script: slave.sh
```

Wrote 14 lines
Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket
Exit Read File Replace Paste Justify Go To Line Redo Copy Where Was

i-01cef55bff55af0e1 (Master)
PublicIPs: 3.80.44.25 PrivateIPs: 172.31.30.100

sudo nano master.sh

Paste the below content to the editor

```
sudo apt update
sudo apt install openjdk-17-jre -y
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian/jenkins.io-2023.key
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt-get update
sudo apt-get install jenkins -y
```

sudo nano slave.sh

Paste the below content to the editor

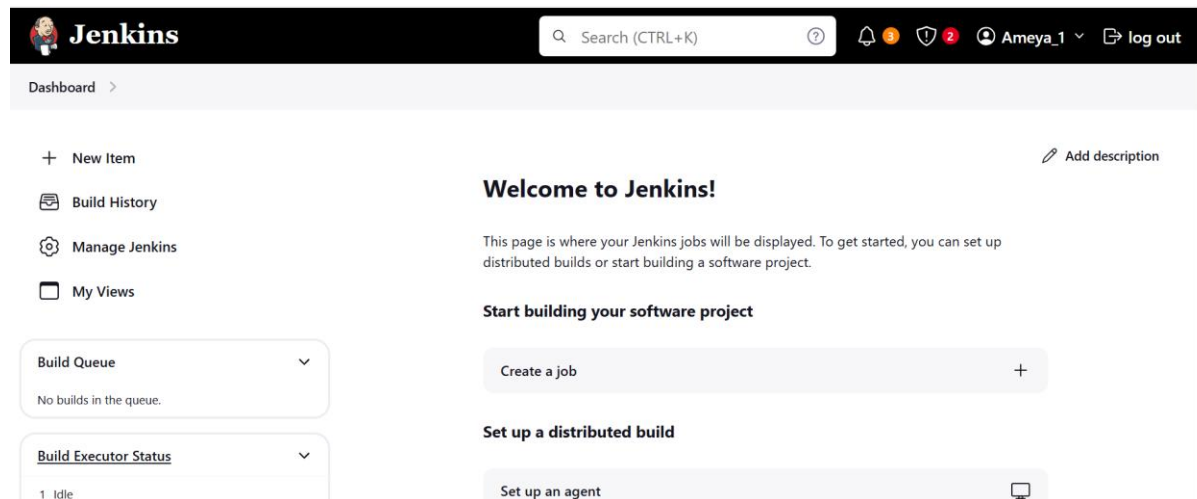
```
sudo apt update
sudo apt install openjdk-17-jre -y
sudo apt install docker.io -y
```

ansible-playbook play.yaml --syntax-check

playbook: play.yaml

ansible-playbook play.yaml --check

○

The image shows the Jenkins web interface. At the top is a black header with the Jenkins logo, a search bar, and user information. Below the header is a light gray sidebar with navigation links: 'New Item', 'Build History', 'Manage Jenkins', and 'My Views'. The main content area has a 'Welcome to Jenkins!' message and two primary action buttons: 'Create a job' and 'Set up an agent'. On the left, there are two expandable panels: 'Build Queue' (showing 'No builds in the queue.') and 'Build Executor Status' (showing '1 Idle').

Jenkins Search (CTRL+K) Ameya_1 log out

Dashboard >

+ New Item

Build History

Manage Jenkins

My Views

Build Queue ▼

No builds in the queue.

Build Executor Status ▼

1 Idle

Welcome to Jenkins! Add description

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Set up a distributed build

Set up an agent

```
ubuntu@ip-172-31-31-138:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~22.04.1
ubuntu@ip-172-31-31-138:~$
```

i-0108991f5ea496174 (Test)

PublicIPs: 100.26.52.202 PrivateIPs: 172.31.31.138

```
ubuntu@ip-172-31-24-74:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~22.04.1
ubuntu@ip-172-31-24-74:~$
```

i-0a185149625988977 (Prod)

PublicIPs: 98.84.188.159 PrivateIPs: 172.31.24.74

Manage Jenkins -> Under System Configuration -> Nodes

New Node -> slave-1 -> permanent Agent -> Create

Then fill required fields with below content

Name: slave-1

Description:

Executors:

Remote root directory: /home/ubuntu/jenkins/

Launch Method: Launch agent via SSH

Private IP of slave1

Add Credentials

Kind: SSH Username with private key

Username: ubuntu

Private key: select Add directly

Paste the content from Launch pem file(Downloaded while creating EC2)

Add

Host Key Verification Strategy: Not verifying Verification Strategy

Save

Dashboard > Manage Jenkins > Nodes >

Usage ?
Use this node as much as possible

Launch method ?
Launch agents via SSH

Host ?
172.31.31.138

Credentials ?
ubuntu

+ Add

Save

You should see the Node added

Repeat the same steps from New Node for slave2

Jenkins

Dashboard > Nodes >

Nodes

+ New Node Configure Monitors

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	3.77 GiB	1 0 B	3.77 GiB	0ms
	slave1	Linux (amd64)	In sync	4.31 GiB	1 0 B	4.31 GiB	15ms
	slave2	Linux (amd64)	In sync	4.31 GiB	1 0 B	4.31 GiB	19ms
Data obtained		0.33 sec	0.33 sec	0.33 sec	0.33 sec	0.33 sec	0.33 sec

Build Queue
No builds in the queue.

Build Executor Status

- Built-In Node 0/2
- slave1 0/1
- slave2 0/1

Login to Individual GitHub

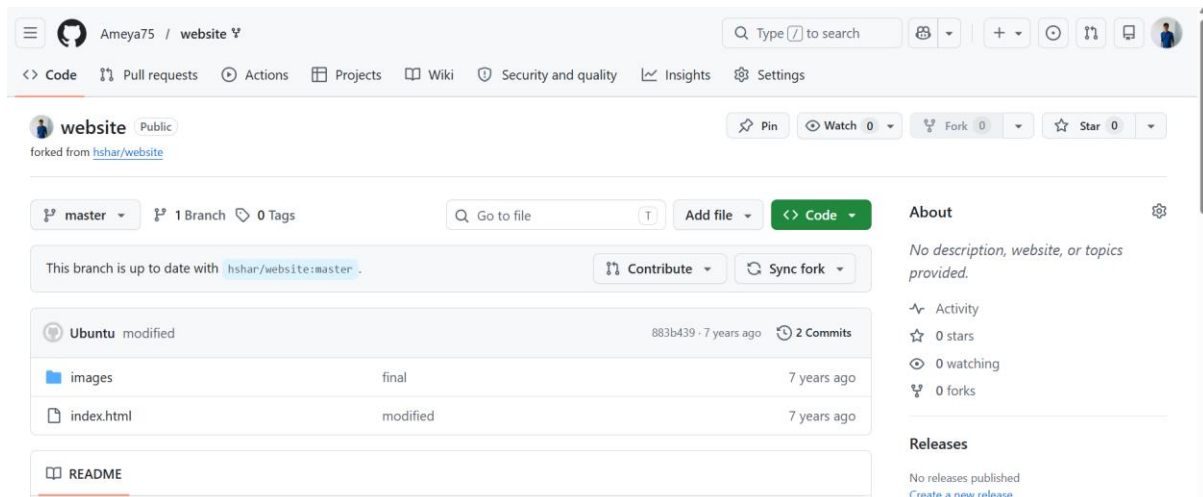
Open the link: <https://github.com/hshar/website>

Click Fork (This copies the code to your repository)

Give your own name or use the same name for the repository (<GitHubUsername/> website)

Click on "Create Fork"

Then in URL, you can see that you've repository under your username



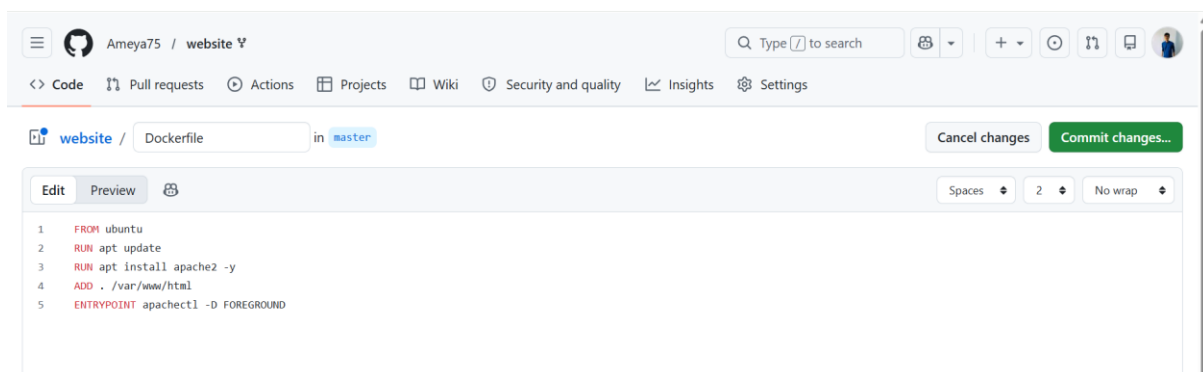
Open the repository and click on Add file -> Create a new file (Like in below screen)

File Content is like: ()

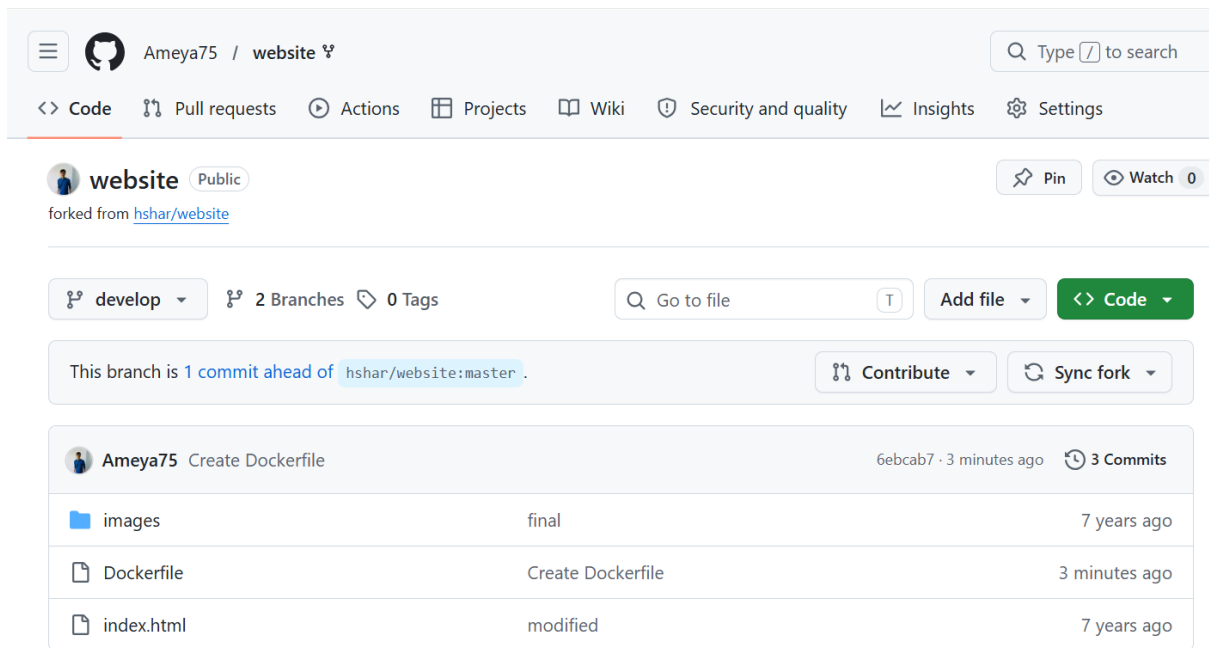
```
FROM ubuntu
RUN apt update
RUN apt install apache2 -y
ADD . /var/www/html
ENTRYPOINT apache2ctl -D FOREGROUND
```

Click “Commit changes” ->(You can change the commit message if you want) “Commit changes”

You should see the Dockerfile in the repository



Create a branch: (as shown in below image)

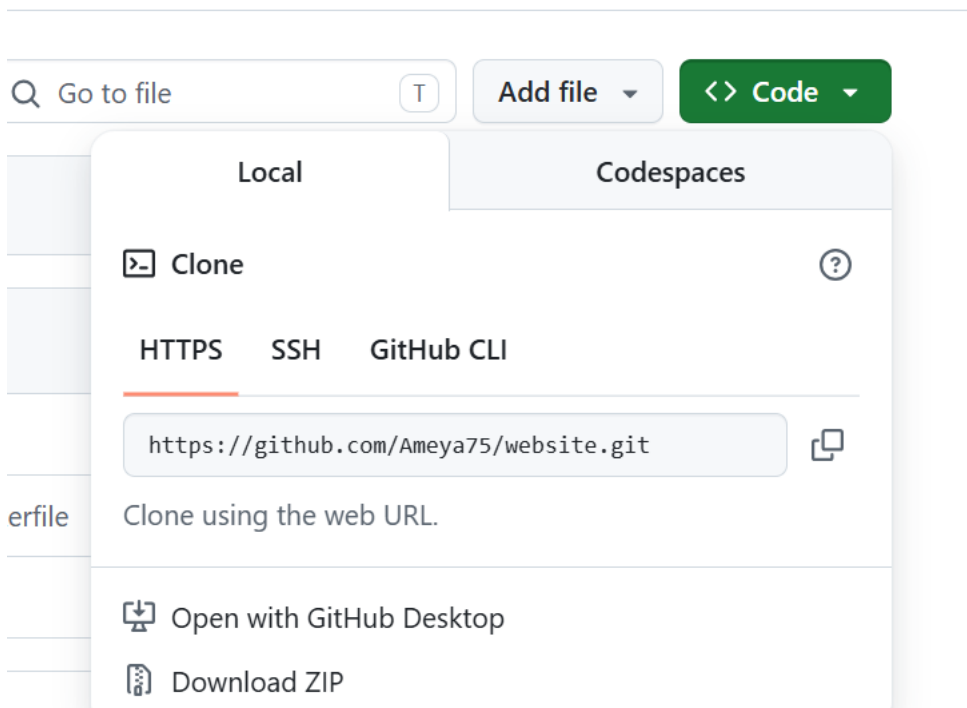


Click “Create branch develop from master”

Move to Jenkins Dashboard in browser

New Item -> Enter Item Name as job1 -> select “Free style project” -> Click “OK”

Select GitHub Project -> Then copy HTML link as shown in image below:



Paste the URL in jenkins tab

Select/Check: Restrict where this project can be run: type or select “slave1” (shown in image)

Dashboard > Job1 > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

GitHub project

Project url ?

https://github.com/Ameya75/website.git

Advanced ▾

- ☐ This project is parameterized ?
- ☐ Throttle builds ?
- ☐ Execute concurrent builds if necessary ?
- ☒ Restrict where this project can be run ?

Label Expression ?

slave1

Then go to Source Code Management and select git

☐ None

☒ Git ?

Repositories ?

Repository URL ?

Credentials ?

+ Add

Advanced ▾

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

Add Branch

Then move to section Triggers -> select “GitHub hook trigger for GITScm polling”

Dashboard > Job1 > Configuration

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Build Steps
- Post-Build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

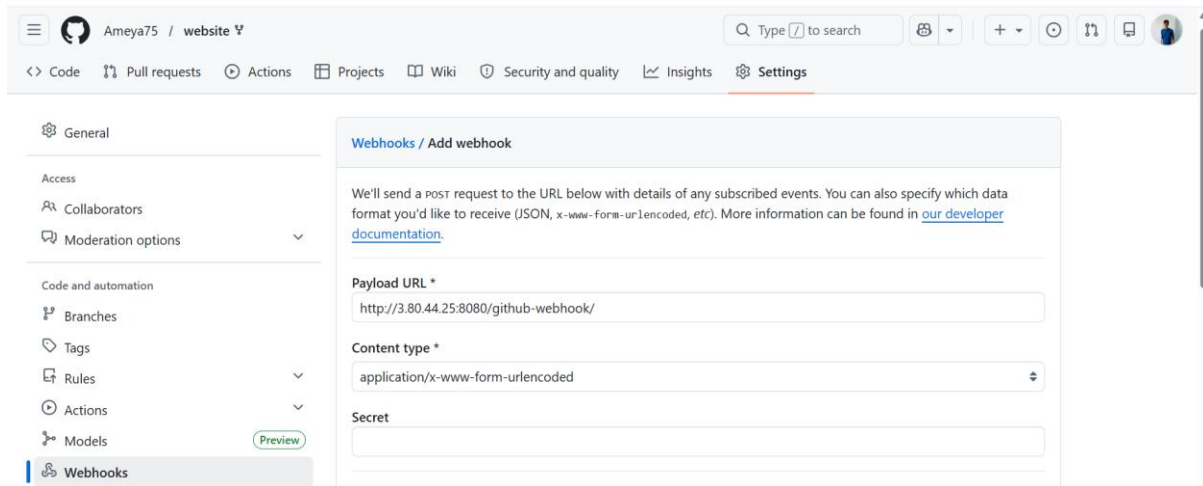
- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Environment

Then Move to github tab

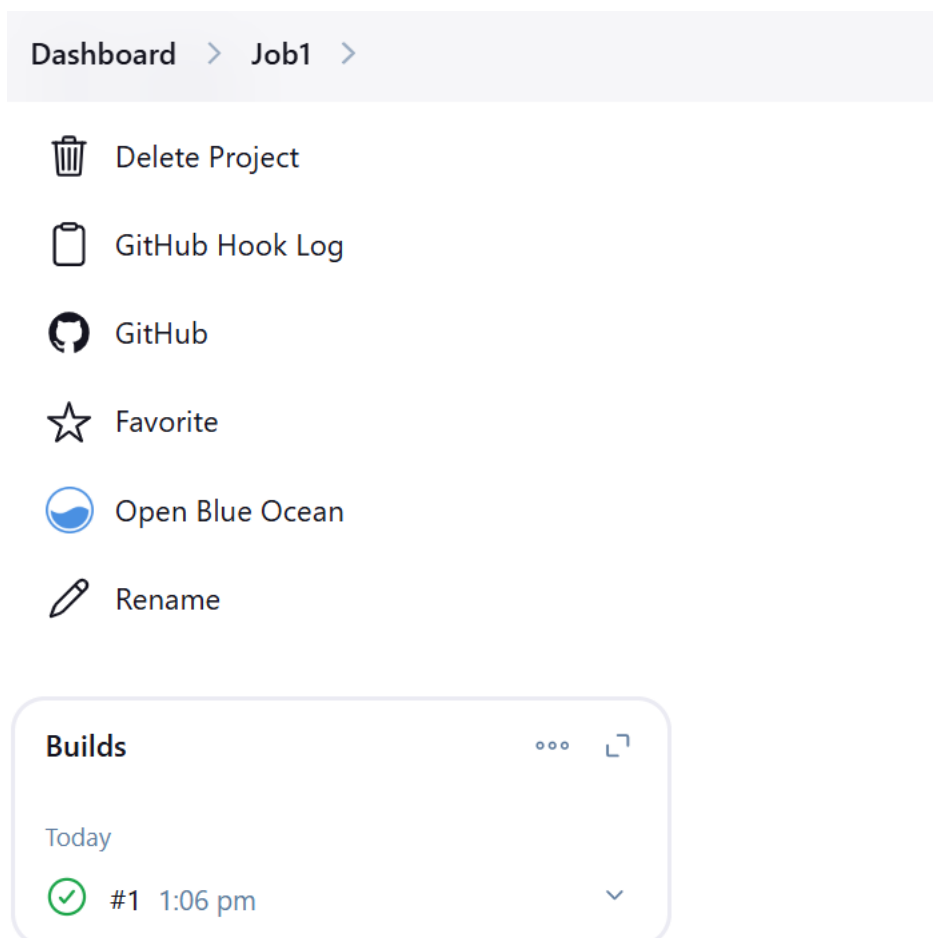
Click “webhooks” under your repository settings -> “Add webhook” -> enter Payload URL:

<http://<PublicIP of Jenkins>:8080//github-webhook/>



Move to Jenkins Tab -> “Save” the Job

You should see Build “Success”



```

Expanded Security Maintenance for Applications is not enabled.

18 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Sun Apr 19 11:55:12 2026 from 18.206.107.29
ubuntu@ip-172-31-31-138:~$ ls
jenkins  jenkins-agent
ubuntu@ip-172-31-31-138:~$ cd jenkins
ubuntu@ip-172-31-31-138:~/jenkins$ ls
remoting  remoting.jar  workspace
ubuntu@ip-172-31-31-138:~/jenkins$ cd workspace/
ubuntu@ip-172-31-31-138:~/jenkins/workspace$ ls
Job1
ubuntu@ip-172-31-31-138:~/jenkins/workspace$

```

i-0108991f5ea496174 (Test)

PublicIPs: 100.26.52.202 PrivateIPs: 172.31.31.138

Click on “Configure” -> “Build Steps” on Left side

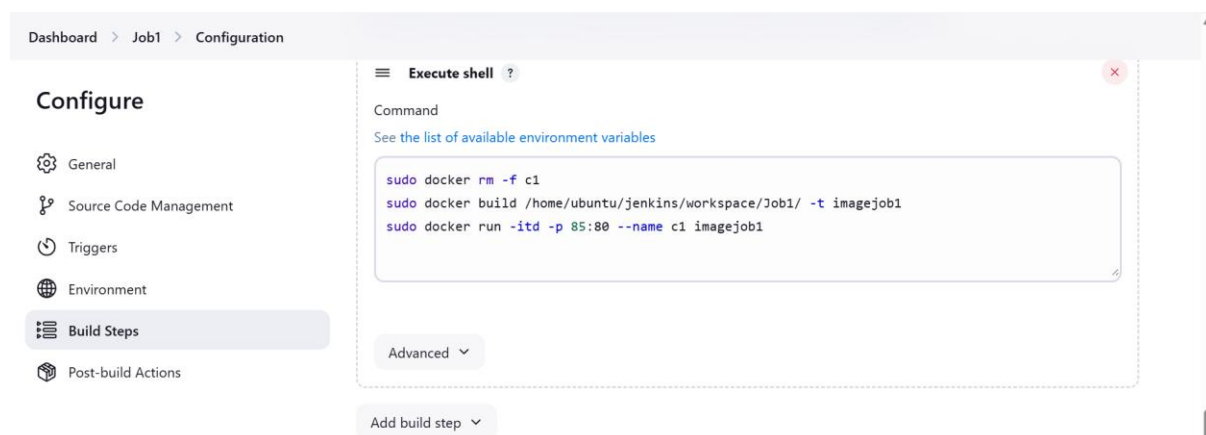
In the editor:

```

sudo docker rm -f c1
sudo docker build /home/ubuntu/jenkins/workspace/Job1/ -t imagejob1
sudo docker run -itd -p 85:80 --name c1 imagejob1

```

Then Click “Save” -> “Build Now”



Copy the Public IP of Test Node and paste it in the browser followed by port 85 as mentioned in build steps.

http://<public_IP_Test_Node>:85/

Should be able to see the website page in the browser.

http://100.26.52.202:85/

